

Smart Prosthetics ERP System Architecture

Technical reference for IT, deployment, and integration · Version 1.1 · August 2026

Client / Center: _____

Server IP (LAN): _____

Document date: _____

Stack: Laravel 10 · PHP 8.x · MySQL/PostgreSQL

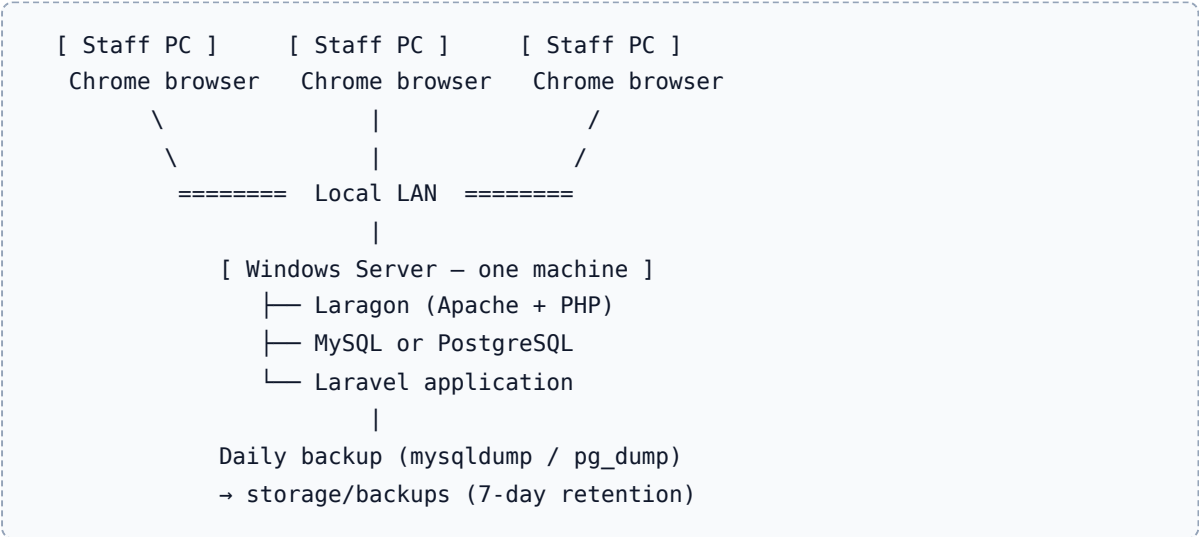
Model: Monolithic offline LAN deployment

Prepared by: Fratelanza

1. Purpose of This Document

This architecture document describes **how the system is built and deployed**: server topology, software layers, database design, security controls, and operational procedures. It complements the *System Blueprint* (business view) and is intended for IT staff, vendors, and technical auditors.

2. Deployment Topology (Offline LAN)



Component	Role	Notes
Staff workstations	Browser clients only	No software install; bookmark server URL
Application server	Hosts web app + database	Dedicated PC; UPS recommended; static LAN IP
Apache	HTTP web server	Serves <code>public/</code> as document root
PHP 8.x	Application runtime	Requires <code>pdo_mysql</code> or <code>pdo_pgsql</code> and <code>ext-zip</code> (Excel import/export via OpenSpout)
Database	Persistent storage	MySQL/MariaDB (Laragon default) or PostgreSQL
Internet	Not required for daily ops	Optional for Git updates or remote support only

Zero cloud dependency: Telegram, Firebase push, and external OCR are disabled in production `.env`. All

patient data remains on-premises.

3. Technology Stack

Layer	Technology
Backend framework	Laravel 10 (PHP 8.1+)
Database	PostgreSQL (documented default) or MySQL/MariaDB
Authentication	Session-based (web guard) + CSRF protection
Frontend	Server-rendered Blade templates + vanilla JavaScript
CSS	Tailwind (pre-built offline assets in production)
QR generation	simplesoftwareio/simple-qrcode
Excel export	OpenSpout
Queue	<code>sync</code> driver (inline — no Redis required offline)
Cache / sessions	File driver (default offline configuration)

4. Layered Architecture

PRESENTATION – Blade views + role dashboards (Chrome)
HTTP – Routes → Controllers (thin – views/JSON only)
DOMAIN – Services (~90 classes) – ALL business logic
DATA – Eloquent Models + PostgreSQL/MySQL
CROSS-CUTTING – AuditService, WorkflowService, Events

Non-negotiable rule

Controllers return views or JSON. **All business rules live in Services.** Workflow stage changes happen only through `WorkflowService` triggered by domain events — never by ad-hoc controller updates.

5. Request Flow Example

```
Browser POST /technical/bom/scan-dispense
→ Route (routes/web/technical.php)
→ BomController::scanDispense()
→ BomService + BarcodeValidationService
→ WorkflowService (BomDispensed event)
→ AuditService::log()
→ StockMovement + CaseRecord updated in DB
→ JSON response to browser
```

6. Directory Structure (Simplified)

Path	Contents
app/Services/	Domain business logic (Patient, Case, Stock, Pricing, Workflow...)
app/Models/	52 Eloquent models (Patient, CaseRecord, Bom, StockItem...)
app/Http/Controllers/	Thin controllers grouped by department
routes/web/	Per-role route files (reception, doctor, admin...)
config/dashboards.php	Role pages, navigation, guards
resources/views/	Blade templates per role dashboard
database/migrations/	Schema version history
storage/backups/	Automated DB dumps (7-day retention)

public/	Web root — Apache document root must point here
---------	---

7. Role-Based Routing

After login, user is redirected to `{role_slug}` (e.g. `/reception` , `/admin`). Middleware enforces dashboard access. Each role has isolated routes defined in `config/dashboards.php` .

8. Entity Relationship Overview

Patient (EMR file, QR card)

- └─ Appointments
- └─ MedicalRecords
- └─ CaseRecord (cases) – workflow root
 - └─ TechOrderSpec (technical specification)
 - └─ PricingRequest → Quote → ApprovalContract
 - └─ Bom → BomItems → StockMovements (barcode)
 - └─ Payments, CreditNotes, ReturnNotes
 - └─ ContractCompany / MilitaryDebt (financial path)

StockItem (catalog, WAC, barcode)

- └─ StockMovements (in/out)
- └─ StockItemPrice (purchase history)
- └─ StockCategory, Supplier

ContractCompany → Patients, Cases, ContractCompanyDebt

User → Role → Permission defaults

AuditLog → append-only (all modules)

9. Key Tables by Domain

Domain	Primary tables
Patients & clinical	patients, appointments, medical_records
Operations	cases, tech_order_specs, case_recommendations
Pricing & quotes	pricing_requests, quotes, quote_items, approval_contracts
Inventory	stock_items, stock_movements, boms, bom_items
Finance	payments, contract_company_debts, military_debts, credit_notes
Master data	contract_companies, suppliers, roles, users
Governance	audit_logs (append-only)

10. Path Isolation Fields

Field	Model	Values
patient_type	Patient	civilian · military
path	CaseRecord	standard (civilian) · military

All BI queries, financial posting, and report filters scope by these fields. Mixing paths in a single query without explicit filter is prohibited.

11. Database Configuration

Connection is defined in `.env` (never committed to Git):

```
DB_CONNECTION=mysql | postgresql
DB_HOST=127.0.0.1
DB_PORT=3306 | 5432
DB_DATABASE=prosthetics
DB_USERNAME=prosthetics_user (recommended – not root in production)
DB_PASSWORD=***
```

12. End-to-End Data Flow



13. Multiple Price Storage Model

Table / field	Role
<code>stock_items.price</code>	Base catalog reference price
<code>stock_items.wac</code>	Current weighted average — updated on every receipt
<code>stock_item_prices</code>	One row per purchase batch: supplier, invoice, amount, qty, received_at
<code>pricing_request_items.unit_price</code>	Snapshoted highest price at calculation time
<code>pricing_request_items</code> (internal)	WAC unit stored in calculation payload / <code>internal_total</code> on request
<code>cases.internal_cost</code>	Sum of WAC-based cost — used in profitability report
<code>cases.quote_total</code>	Revenue basis for civilian profitability

14. WAC Recalculation (Moving Average)

```
StockPriceService.recalcWac()  
new_wac = (prior_qty × prior_wac + in_qty × in_price) / (prior_qty + in_qty)
```

Triggered by:

- StockReceiveService (physical inward)
- StockPriceService.addBatch() (price batch registration)

NOT used for civilian quote line pricing – only for inventory value & profit cost.

15. Stock Deduction on Sale / Dispense

There is no separate “sales module” that manually adjusts stock. **Dispense IS the stock deduction** — tied to a validated work order:

1. BomService.refreshUnitCostsAtDispense() — current WAC stamped on every BOM line
2. BomService validates scan against BomItem line
3. stock_items.qty decremented; reserved released if applicable
4. StockMovement created (type issue, unit_cost = WAC, BOM reference, balance_after)
5. cases.issue_cost snapshot stored ($\Sigma \text{qty} \times \text{WAC}$)
6. WorkflowService fires BomDispensed event
7. FinancialPostingService.postOnDispense() — civilian contract debt (atomic ledger_posted_at)

Mismatch scan → BarcodeDispenseMismatchException → transaction blocked → audit entry. Stock and finance stay aligned.

Financial Services & Reporting Layer

Accounting profile implementation

Architecture
5 / 8

16. Financial Services Map

Service	Responsibility
StockPriceService	WAC recalc, highestUnitPrice(), wacUnitPrice(), price batches
PricingService	Dual calculation: computed_total + internal_total per pricing request
CostingService + CostingEngine	Overhead components + profit rate → selling_price
FinancialPostingService	Civilian debt on dispense; military debt on delivery
ContractDebtService	Increase/decrease contract entity due balances
FinancialBalanceService	Opening / movement / closing for cash, civilian, military, inventory domains
ProfitabilityReportService	Revenue – issue_cost (WAC at dispense) = margin per case
InventoryFinancialReconciliationService	Period bridge: stock issue WAC vs delivered revenue & margin
ItemPricingAnalyticsService	Per-item WAC, highest/lowest batch price, unit margin %
BiReportService	Five BI boards + WAC/highest inventory valuation spread
AdminReportsHubService	Profitability, reconciliation, item margins, inventory valuation, movements

17. Admin Reports (Finance Department)

Report section	Route	Key output
Opening balance	/admin/reports/opening-balance	Cash, civilian AR, military AR, inventory WAC — start of period
Closing balance	/admin/reports/closing-balance	Same four domains — end of period after movements
Profitability review	/admin/reports/profitability	Per case: revenue, issue_cost (WAC), margin, margin %
Inventory ↔ Finance reconciliation	/admin/reports/inventory-reconciliation	Receive/issue WAC values vs delivered revenue & gross margin
Item profit margins	/admin/reports/item-margins	WAC, highest/lowest price, unit margin % per SKU

Inventory valuation	/admin/reports/inventory-valuation	Per item: WAC value + highest-price value + purchase batches
Stock movements	/admin/reports/inventory-overview	Qty, WAC unit cost, line value, BOM reference
Civilian debts	/admin/reports/civilian-debts	Due, collected, remaining per entity
BI Board 5	/admin/overview	WAC vs highest purchase price — margin erosion flag

18. Profit Margin Calculation

At barcode dispense (BomService.releaseToWip):

- refreshUnitCostsAtDispense() → all BOM lines use current WAC
- StockMovement.unit_cost = WAC
- cases.issue_cost = $\sum(\text{qty} \times \text{WAC})$ ← snapshot for finance

ProfitabilityReportService (delivered cases):

```
revenue = quote_total (civilian) | military_selling_price (military)
cost    = issue_cost if set, else internal_cost (WAC estimate at costing)
margin  = revenue - cost
margin_pct = margin / revenue × 100
```

ItemPricingAnalyticsService (catalog):

```
unit_margin = highest_purchase_price - WAC
margin_pct  = unit_margin / WAC × 100
```

19. Accounting Period Model

AccountingPeriod + PeriodOpeningOverride allow finance to set manual opening balances at period start (e.g. fiscal year alignment). FinancialBalanceService.balancesForPeriod() applies overrides when generating opening/closing reports.

Workflow Engine & Domain Services

Event-driven state machine

Architecture
4 / 6

12. WorkflowService — Single Authority

All case stage transitions (`stage_key` , `manufacturing_stage`) are owned by `WorkflowService` . Domain services emit workflow events; the engine applies transitions inside a database transaction with audit logging.

13. Workflow Events

Event	Triggered by service	Typical transition
ExamApproved	MedicalRecordService	exam → technical
SpecSaved	SpecService	technical → adjustments / pricing
AdjustmentsCompleted	AdjustmentsService	→ cost_calc
CostingCompleted	CostingService	→ quote (civilian) or operations (military)
QuoteIssued	QuoteService	→ awaiting approval
OperationsApproved	OperationsService	→ manufacturing
BomDispensed	BomService	warehouse complete → production department WIP
BomFinished	BomService	→ ready_delivery
Delivered	DeliveryService	→ closed + financial posting

14. Key Domain Services

Service	Responsibility
PatientService / CaseService	Registration, case creation, path assignment
SpecService / CostingService	Technical spec, adjustments & costs, approval (cost confirmation)
WorkshopAssignmentService / WorkshopTrackingService	Technician & section assignment, per-technician progress board (production department)
ReturnNoteService	Material returns — stock restored at WAC matching dispense; <code>issue_cost</code> sync
PricingService / QuoteService	Background pricing, quote PDF + QR
StockReceiveService	Inward stock, WAC recalculation
BomService + BarcodeValidationService	BOM dispense, scan validation, block on mismatch
FinancialPostingService	Contract debt (civilian) / military debt posting
BiReportService	Five read-only BI command boards
AuditService	Append-only audit log writer

PatientDataPurgeService	Safe purge of patient data (keeps master data)
-------------------------	--

15. Pathway Configuration

Admin can configure optional stage skips (e.g. skip exam, auto-skip cashier) via `PathwayConfigService` , `PathwayStep` , and `WorkflowStagePolicy` — without changing core event definitions.

16. Security Controls

Control	Implementation
Authentication	Session login; idle timeout (default 5 min); per-user accounts
Authorization	Role-based dashboard routing; service-layer visibility rules
CSRF	Laravel CSRF tokens on all POST requests
Production debug	<code>APP_DEBUG=false</code> — no stack traces to users
HTTPS cookie	<code>SESSION_SECURE_COOKIE=false</code> on HTTP LAN; true only with HTTPS
External notifications	<code>TELEGRAM_ERROR_NOTIFY=false</code> , <code>FIREBASE_ENABLED=false</code>
Firewall	Apache allowed on Private network only; port 80 not exposed to internet
Audit integrity	<code>AuditLog</code> model blocks update/delete at ORM level

17. Environment File (.env)

The `.env` file holds server-specific secrets and is **never** stored in Git. Critical production values:

```
APP_ENV=production
APP_DEBUG=false
APP_URL=http://<server-lan-ip>
DB_* = database credentials
SESSION_LIFETIME=5
SESSION_IDLE_TIMEOUT=5
```

18. Backup Strategy

Item	Detail
Command	<code>php artisan prosthetics:backup</code>
Engine	<code>mysqldump</code> or <code>pg_dump</code> (auto-detected from DB driver)
Location	<code>storage/backups/db-YYYY-MM-DD_HHMMSS.sql.gz</code>
Retention	7 days (configurable via <code>--keep=N</code>)
Schedule	Windows Task Scheduler — daily at 02:00 AM recommended
Off-site copy	Weekly USB / secondary disk copy of <code>storage/backups</code>

19. Update Procedure (Offline)

1. Run backup: `php artisan prosthetics:backup`
2. Pull code: `git pull` (or copy from USB if no internet)

3. Dependencies: `composer install --no-dev --optimize-autoloader` (enable PHP `ext-zip` first on Laragon)
4. Schema: `php artisan migrate --force`
5. Cache: `php artisan config:cache && route:cache && view:cache`
6. Staff refresh browser: Ctrl+F5

Never overwrite `.env` or `storage/` during updates.

20. Useful Maintenance Commands

Command	Purpose
<code>prosthetics:backup</code>	Database backup
<code>prosthetics:purge-patient-data --force</code>	Remove all patient/case data; keep users & master data
<code>prosthetics:purge-logs</code>	Clear application log files
<code>config:clear</code>	After <code>.env</code> changes before re-caching

Network Access & Integration Summary

How staff connect and what stays local

Architecture
6 / 6

21. Staff Access Pattern

```
Server static IP: http://192.168.x.x
Local test:      http://127.0.0.1
Laragon alias:   http://prosthetics.test (server only – hosts file)
```

Staff PC requirements:

- Google Chrome (recommended)
- Connected to center LAN
- Bookmark / desktop shortcut to server IP
- No application installation

22. Apache Document Root

For correct routing, Apache must serve the Laravel `public/` folder:

```
DocumentRoot → C:/laragon/www/prosthetics/public
```

After IP change, update `APP_URL` in `.env` and run `php artisan config:cache`.

23. Optional / Disabled Integrations

Integration	Status in offline production
Firebase push notifications	Disabled — local polling fallback
Telegram error alerts	Disabled — prevents data leaving LAN
Cloud OCR / external APIs	Not used — local OCR where implemented
Redis / Memcached	Not required — file drivers suffice
Email (SMTP)	Optional — not required for core workflow

24. Scalability & Limits

- **Designed for:** Single center, one LAN, tens of concurrent browser sessions
- **Not designed for:** Multi-site cloud SaaS, public internet exposure, mobile native apps
- **Horizontal scaling:** Not applicable — single-server offline model
- **Performance:** Route/config/view caching enabled in production; OPcache recommended on PHP

25. Related Documents

Document	Audience
System Blueprint (English)	Management, department heads — business workflows
System Architecture (English)	IT — deployment, layers, security

docs/downloads.html	Printable PDF pack — blueprint, architecture, deploy guide
Deploy Manual (PostgreSQL / Windows)	IT installer — step-by-step setup
Deployment Checklist (printable)	On-site installation sign-off

IT contact: _____

Support vendor: Fratelanza

Approved by: _____

Signature / date: _____